

Declaration

I declare that the work which follows is my own, and that any quotations from any sources (e.g. books, journals, the internet) are clearly identified as such by the use of 'single quotation marks', for shorter excerpts and identified italics for longer quotations. All quotations and paraphrases are accompanied by (date, author) in the text and a fuller citation is the bibliography. I have not submitted the work represented in this report in any other course of study leading to an academic award.

Student..... *Gráinne O' Connor* Date30/3/2026.....
Work Place Mentor.....N/A..... Date

Acknowledgements

I would like to express my sincere thanks to my supervisor for their guidance, feedback, and support throughout this project. I am also grateful to my lecturers and classmates for their advice and encouragement during the development and evaluation of the work. Finally, I would like to thank my family and friends for their patience and support throughout the process.

Title



“TL;DR: A comparison of container image vulnerability scanners in a CI/CD pipeline deploying an AI risk assistant for online agreements”

Name: Gráinne O' Connor
Student number: 11402918
Supervisor: Peter Windle

Submission date: 30th March, 2026

Course: Higher Diploma in Computer Science,
South East Technological University, Waterford.

Abstract

This project investigates the use of container vulnerability scanning within a GitOps-based deployment workflow by comparing Trivy and Grype. A small FastAPI application was containerised, stored in GitHub Container Registry, and deployed to a local Kubernetes cluster using Helm and ArgoCD, while both scanners were run in GitHub Actions across multiple container images. The study evaluated detection quality, overlap, actionability, and developer experience using a combination of quantitative results and manual inspection of sample findings. The results suggest that the two scanners differ in noise, reporting style, and remediation guidance, with each offering distinct strengths depending on whether the priority is coverage, clarity, or ease of integration. Overall, the project shows that vulnerability scanning is most useful when combined with a reproducible deployment pipeline and a clear method for interpreting findings.

Table of Contents

Declaration	1
Acknowledgements	2
Title	3
“TL;DR: A comparison of container image vulnerability scanners in a CI/CD pipeline deploying an AI risk assistant for online agreements”	3
Abstract	4
Table of Contents	5
Table of Tables	8
Table 1: Stakeholders and personas relevant to this research area	8
Table 2: Functional and non-functional requirements	8
Table 3: Container image sample set	8
Table 4: Total number of vulnerabilities, severities, and overlap of Trivy and Gripe	8
Table 5: Sample set of overlap vulnerabilities	8
Table 6: Actionability scores of Trivy and Gripe on a sample set of vulnerabilities	8
Table of Figures	9
Glossary	10
Introduction	12
Background	12
Problem Definition	13
Solution	13
Project Goal	13
Research Question	13
Research and Analysis	14
Stakeholders and Personas	14
Functional and Non-functional Requirements	15
Scope	16
Out of scope/Constraints	16
Features and Market Research	17
Vulnerability Image Scanners	17
Container image sample set	18
Why Trivy and Gripe for this project	19
Modelling	20
System Context Diagram	20
CI/CD Pipeline Model	21
Data and Reporting Model	22
Planning	24
Technology selection	24
Github Actions	24
GitHub Container Registry	24

kind	24
Helm	24
ArgoCD	25
Trivy and Grype	25
Phased Plan	25
Implementation	26
Application Implementation	26
Dockerfile	27
Container Registry and Image Management	27
Kubernetes and Helm	27
ArgoCD and GitOps Setup	27
CI Workflows	28
Security Scanning	29
Findings	30
Detection quality tests	30
Actionability tests	30
Developer experience tests	31
Quantitative Findings	31
Qualitative Findings	33
Configuration and integration complexity	33
Noise and predictability	33
Clarity and remediation guidance	35
Table 6: Actionability Scores of Trivy and Grype on a sample-set of vulnerabilities.	37
Scan Times	37
Reflection	38
What Worked Well	38
GitOps with ArgoCD and Helm	38
Using CI for scanning and comparison	38
Persona-driven requirements	38
Challenges and Limitations	38
GHCR authentication	38
Automated ArgoCD sync	39
Limited ground truth for vulnerability findings	39
Time constraints and scope	39
Future Work	39
DAST or runtime scanning	39
Add more scanners	40
Metrics and dashboards	40
SBOM generation	40
References	41

Appendices	43
AI use	43

Table of Tables

Table 1: Stakeholders and personas relevant to this research area

Table 2: Functional and non-functional requirements

Table 3: Container image sample set

Table 4: Total number of vulnerabilities, severities, and overlap of Trivy and Grype

Table 5: Sample set of overlap vulnerabilities

Table 6: Actionability scores of Trivy and Grype on a sample set of vulnerabilities

Table of Figures

Figure 1: High-level system context diagram

Figure 2: CI/CD pipeline model diagram

Figure 3: Scan workflow sequence in GitHub Actions

Figure 4: Workflow steps to summarise and normalise scan results

Figure 5: UI of 'Terms Long; Didn't Read' FastAPI/Llama3 web application

Figure 6: ArgoCD dashboard showing tldr-app in healthy and synced state

Figure 7: Sequence of jobs in the tldr-app GitHub Actions workflow

Figure 8: Snippet of Trivy JSON report (not normalised)

Figure 9: Comparison results in the GitHub Actions workflow view

Figure 10: Normalised Gype scan results for debian:12

Figure 11: Normalised Trivy scan results for debian:12

Figure 12: Duplication of Trivy results for CVE-2026-3184

Figure 13: Debian package 'zlib' affected by CVE-2023-45853

Figure 14: National Vulnerability Database entry for CVE-2023-45853

Figure 15: Screenshot of image scan times in GitHub Actions

Glossary

Actionability

How useful a vulnerability finding is for taking practical next steps, based on clarity, remediation advice, and prioritisation support.

ArgoCD

A GitOps tool for Kubernetes that continuously compares the live cluster state with the desired state defined in Git and applies changes when needed.

CRD (Custom Resource Definition)

A Kubernetes extension that lets you define and manage custom resource types beyond the built-in objects.

DAST (Dynamic Application Security Testing)

Security testing performed against a running application to identify issues that may not be visible in static code or image analysis.

Drift

A difference between the configuration stored in Git and the actual state of the running cluster.

False negative

A real vulnerability that is not reported by a scanner or is missed in a comparison.

False positive

A vulnerability reported by a scanner that is not actually relevant or exploitable in the context being tested.

GHCR (GitHub Container Registry)

GitHub's container registry service for storing and distributing container images.

GitOps

An operational model where Git is treated as the source of truth for infrastructure and deployment configuration.

Grype

An open-source container vulnerability scanner that identifies known vulnerabilities in software packages and container images.

Helm chart

A packaged set of Kubernetes manifests and configuration templates used to deploy applications consistently.

Kind

A tool for running local Kubernetes clusters using containers, commonly used for development and testing.

Namespace

A Kubernetes mechanism for logically separating resources within a cluster.

Overlap

The set of vulnerabilities reported by both scanners, used to compare how much their findings agree.

Severity

A rating that indicates how serious a vulnerability is, commonly grouped as Critical, High, Medium, or Low.

Self-heal

An ArgoCD feature that automatically corrects cluster drift by reapplying the desired state from Git.

Trivy

An open-source security scanner used to detect vulnerabilities in container images and other software artefacts.

Introduction

Background

Containerization is the popular mode of deploying multiple applications without rewriting the program code (AWS, 2025). Container images are the “foundation” for running containers. They typically contain the “application code, runtime, libraries, environment variables, and other dependencies required to run an application in isolation” (Armo, 2025).

The landscape of containerization has expanded and evolved significantly since its inception with Docker in 2013. With very few “mission-critical production deployments” in the early days, developers used it primarily for testing and staging applications (Tozzi, 2019).

Today, the picture is very different. With the increased adoption of containerization in enterprise settings - 90% in 2025, up from 82% in 2024 (Wilson, Diachkova & Floyd, 2025) - there has been an uptick in instances of malware within container images along with it (OWASP, 2025). This has increased the need for tools that can automatically detect vulnerabilities in images before they are deployed.

Docker's 2025 State of Application Development Report reveals that the responsibility for security is being felt by more than just security engineers, stating that when scan alerts go off, “everyone pitches in” making this a pertinent area for research.

Container security is implemented by running vulnerability scanners which are used to detect insecure or misconfigured images before deployment (OWASP, 2025). In many pipelines their findings are used to block deployments that fail compliance checks.

There are two types of container vulnerability scanners:

- Static: These scans occur in “offline mode”, meaning they are concerned with the existing immutable layers of a container image before deployment.
- Dynamic: These are the next phase of vulnerability scanning, concerned with container run-time, monitoring and identifying threats that occur during execution (SentinelOne, 2025).

While each of these scanners do an important job, it is not a seamless, errorless process.

Commonly occurring issues which make SAST tools less reliable are the rates of ‘false positive’ results emerging in scan results. In the case of false positives, the vulnerability scanner has flagged potential vulnerabilities, however its method of doing so has been by code pattern rather

than by exploitability (Kaiuwan, 2026). Therefore, the perceived threat within the static image could be valuable time wasted by developers who then have to triage what turns out to be safe.

Problem Definition

Due to the complexity of matching vulnerabilities to image content, many container vulnerability scanners produce 'false negatives' and 'false positives' in their reports. This often causes disruption to DevOps workflows with time spent investigating non-existent threats, or overlooking real threats that go undetected.

Solution

Shift left is the practice of incorporating security checks into the software development process both early and often. This approach makes vulnerabilities cheaper and easier to fix (Snyk, 2026). As such, testing and evaluating the performance of vulnerability scanners will provide insight into the best options for users of these tools such as Software Engineers, Platform/DevOps engineers, Security/DevSecOps engineers, and Product Managers. This project implements a GitOps-style CI/CD pipeline using GitHub Actions, GHCR, Kubernetes and ArgoCD to embed Trivy and Gype into the build and deployment flow.

Project Goal

This project seeks to compare the performance of Trivy and Gype - two static vulnerability scanners - as cost-effective CI tools to protect registries and clusters. It will compare their performance in the following evaluation framework:

- Detection: number of vulnerabilities, overlap between tools
- Quality: false positives, false negatives, actionability of findings
- Developer Experience: usability, integration with CI,

Research Question

- How do Trivy and Gype compare in detection quality, noise, and developer experience when integrated into a CI/CD pipeline?

Research and Analysis

Stakeholders and Personas

Persona	Role	Goals	Pain points	Needs
Software Developer/Engineer	Writes the code for the application and maintains it.	Get fast feedback when code or dependencies introduce vulnerabilities. Keep pipelines green so features deploy quickly.	Long scan times which block pull requests. “Noisy” reports with inactionable or false-positive	CI gates that are predictable. Clear, actionable vulnerability output
DevOps Engineer	Designs and maintains the pipeline that builds the project.	Standardise how images are built, scanned and deployed to clusters. Minimise manual interventions.	Integrating and maintaining multiple scanners in CI. Balancing stronger security with acceptable pipeline duration.	Clear comparison of vulnerability scanners in terms of reliability, performance and ease of automation.
DevSecOps Engineer	Defines security policies, chooses vulnerability scanning tools, and is responsible for vulnerability management and compliance.	Ensure containers in registries and clusters meet a minimum security baseline. Reduce risk of exploitable vulnerabilities reaching production.	Inconsistent scanner results. High rate of false-positives overwhelming limited security bandwidth.	Which scanner offers better detection. Policy recommendations that are realistic for CI.
Product Owner	Oversees the team delivering the application - responsible for delivery speed and risk exposure.	Maintain sustainable delivery - not constantly blocked by security. Demonstrate to stakeholders that security is built into the process.	Trade-off between stricter gates and missed deadlines. Lack of clear metrics on how security tools impact the duration of pipeline process.	Quantitative data on scan time and failure rates of pipeline. Recommendation on a tool that best balances security with velocity.

Table 1: Stakeholders and Personas relevant to this research area.

Functional and Non-functional Requirements

Functional	Non-functional
• The CI system will build a container image for every commit to the main or feature branch. • The CI system will push successfully built container images to the container registry. • The CI pipeline will run each new image on Trivy and Gype and produce a JSON report as an artifact for both tools. • The pipeline will evaluate the Trivy and Gype results against a configurable vulnerability policy. • The pipeline will fail if any scanner reports one Critical or High vulnerability - in line with current policy. • On successful scans, the pipeline will update the image tag in the manifests repository. • ArgoCD will automatically sync the manifests repository and deploy the updates image to the Kubernetes cluster. • The system will store the scan reports and pipeline logs for at least 30 days for audit and analysis.	• Performance: ◦ Each scanner step will complete within 5 minutes for images under 1GB • Reliability: ◦ Rerunning the same commits without changing configuration will produce the same pass/fail outcome. ◦ The CI pipeline will successfully complete 99% of runs over 2 weeks. • Usability/Developer Experience: ◦ Scan reports will include a clear summary ◦ Developers will be able to identify the affected package and recommended fix from the report in at least 90% of cases. • Security: ◦ All communication with the container registry and Git repo will use TLS ◦ Scans will not expose secrets or sensitive data in logs. • Maintainability: ◦ Scanner configuration (policies, severity thresholds, excludes) will be stored as code in version control. ◦ It will be possible to upgrade Trivy or Gype to a new version by changing a single configuration location (e.g. CI image tag)

Table 2: Non-functional and functional requirements

Scope

- This project will build a small FastAPI + LLM Wrapper demo application packaged as a Docker image.
- It will set up a CI pipeline that builds the image and runs Trivy and Gype as static image vulnerability scanners.
- It will use a single Kubernetes cluster and ArgoCD for GitOps-style deployment.
- The project will implement scan-gates promotion: only images passing defined policies are updated in the manifests repo and deployed.
- It will design a small set of experimental scenarios (e.g. clean base, known vulnerability base, vulnerable dependency) and collect metrics on detection, quality and developer experience.
- The comparison is limited to Trivy and Gype as cost-effective open-source CI-tools.

Out of scope/Constraints

- No runtime/behavioural security
- No commercial or SaaS scanners
- No large-scale performance/load testing across many services.
- No deep governance/compliance beyond basic policies
- Resource: Work will run on limited hardware so images and scans must stay within reasonable size and time limits.
- Tooling: Only open-source scanners (Trivy and Gype) and commonly available CI/CD tooling (e.g. GitHub Actions, Argo CD, Kind/k3s) will be used.
- Environment: Experiments will be conducted on a single Kubernetes cluster, not multiple clouds.
- Knowledge/data: Ground truth about “real” vulnerabilities is approximated using known vulnerable images and the union of scan results, not a formal certification lab.

Features and Market Research

Vulnerability Image Scanners



- **Trivy**

Trivy is an open-source container vulnerability scanning tool owned by Aqua Security and designed to integrate into CI/CD pipelines. It can scan project repositories locally, container images, Helm charts as well as providing monitoring and alert options for Kubernetes cluster management. Its scope of coverage is broad, targeting OS packages, language dependencies, and Infrastructure as Code. It has been described as a “chameleon” (Kelcode, 2024) for its capability to blend into workflows “seamlessly”. In this project, Trivy is selected as one of the comparison tools because it is widely adopted, actively maintained, easy to run in GitHub Actions, and available under a permissive open-source licence.



- **Grype**

Grype is an open-source container and filesystem vulnerability scanner from Anchore, optimised for automation. It can scan container images directly or consume SBOMs generated by companion tools such as Syft, and is designed to fit naturally into CI pipelines. Like Trivy, it focuses on known-vulnerability detection against multiple advisory sources, and emphasises a clean JSON output format along with clear exit-code behaviour for gating builds.



- **Clair**

Clair is another widely cited open-source container vulnerability scanner that performs static analysis of container images stored in registries. It has historically been used as a backend service rather than a lightweight CLI, and its typical deployment model is to run as part of a registry or platform rather than as a simple job step in a GitHub Actions workflow.

Container image sample set

To make the comparison between Trivy and Grype realistic and repeatable, the project uses a small, diverse set of container images:

Image	Type	Reason for inclusion
tldr-app:ci-build	Application image	Real project image built from this repository's FastAPI demo app.
alpine:3.18	Minimal base	Very small, widely used base with few packages and a low attack surface.
debian:12	General-purpose base	Common, full-featured base image with many OS packages and CVEs, representative of "heavier" images.
python:3.14-slim	Runtime image	Typical language/runtime image (Python on a Debian base) used as a building block for application images.

Table 3: Set of Container Images

This mix allows the evaluation to cover: a real application image, a minimal base, a more feature-rich base, and a language runtime. It was intended to test a hardened Python base image from Docker Hardened Images (DHI) in a small side experiment to illustrate how moving from a standard base to a hardened base can reduce vulnerability counts and scanner noise, but was not part of the core four-image comparison set and time did not allow for this in the project report, but will feature in the project video demonstration and presentation.

Why Trivy and Gype for this project

Trivy and Gype are both:

- Open-source and free to run in CI.
- Designed to be invoked as simple command-line tools or GitHub Actions steps.
- Focused on static image analysis (SCA) rather than runtime protection, which aligns with this project's scope.

They therefore provide a fair, like-for-like comparison of two realistic choices a team might adopt to secure their registry and CI/CD pipeline. Clair and commercial or SaaS scanners are intentionally excluded from implementation to keep the work achievable, but are referenced in the background and future-work discussion as potential next steps.

Modelling

System Context Diagram

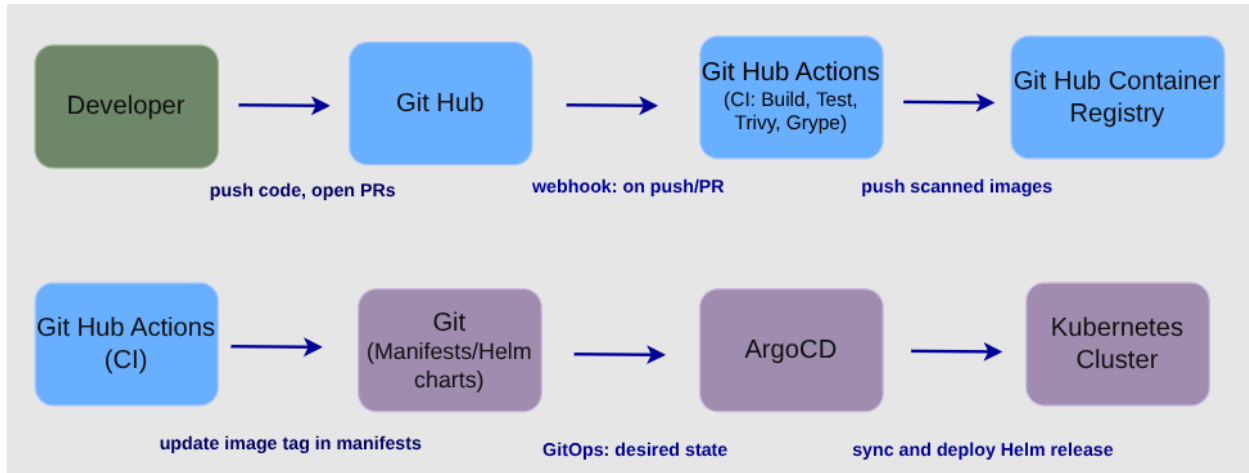


Figure 1: High level system context diagram

The system context diagram in Figure X shows the main actors and platforms in the solution. A developer pushes code changes and pull requests to GitHub, which triggers GitHub Actions as the CI engine. CI builds the application image, runs tests, and executes Trivy and Grype before pushing the scanned image to the GitHub Container Registry (GHCR). In parallel, Git stores the manifests and Helm charts that reference the image tag. A separate GitHub Actions job updates the image tag in the manifests repository when a build passes. ArgoCD continuously watches the manifests Git repository, treats it as the desired state, and reconciles any changes into the Kubernetes cluster, ensuring that only images that have passed the CI security gates are deployed.

CI/CD Pipeline Model

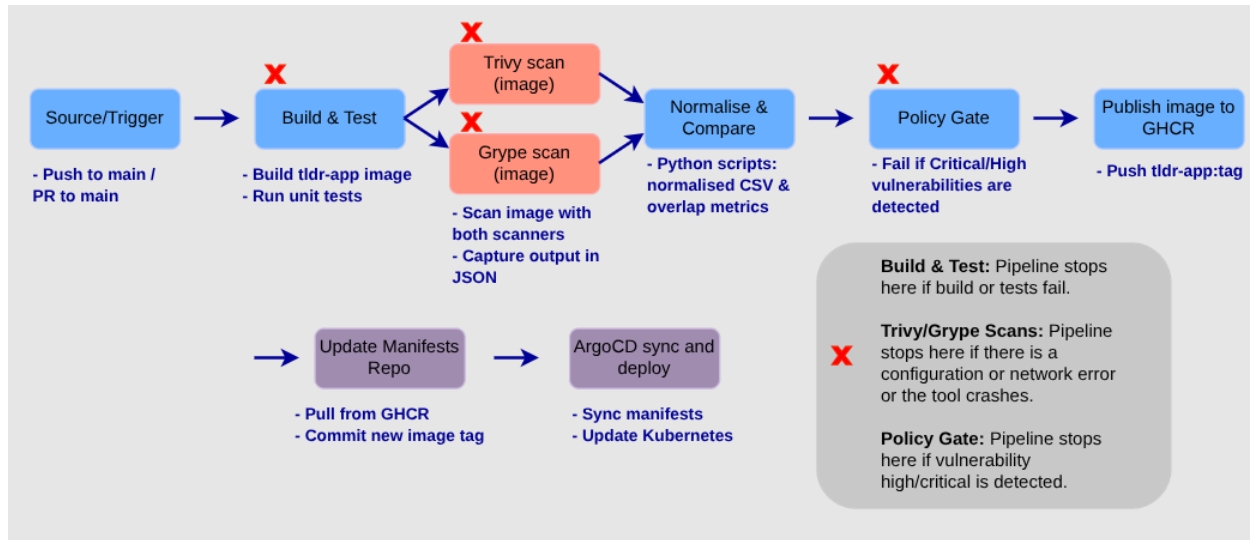


Figure 2: CI/CD pipeline model diagram

The CI/CD pipeline model in Figure X illustrates the stages a change passes through from commit to deployment. A push to the main branch or a pull request to main acts as the source trigger, starting a GitHub Actions workflow. The pipeline first runs a Build & Test stage, which builds the *tldr-app* container image and executes unit tests. If this succeeds, the image is scanned by both Trivy and Grype, with each scanner producing a JSON report of detected vulnerabilities. The Normalise & Compare stage runs Python scripts that convert these reports into normalised CSV files and compute overlap metrics between the tools. A Policy Gate stage evaluates the combined findings and can fail the pipeline if Critical or High vulnerabilities are present. When the gate passes, the image is pushed to GHCR, a separate job updates the image tag in the manifests repository, and ArgoCD then syncs those manifests and deploys the new image to the Kubernetes cluster.

In this project, Trivy and Grype are configured to always return a successful exit code on completed scans (exit code 0), regardless of how many vulnerabilities they find. Their role in the CI pipeline is to produce JSON reports, not to enforce policy directly. A separate Policy Gate stage, implemented in Python, consumes both scanners' normalised outputs and is the place where vulnerability thresholds (for example, failing the pipeline on any High or Critical issue) will be applied centrally in future.

Data and Reporting Model



Figure 3: Screenshot of the scan workflow sequence

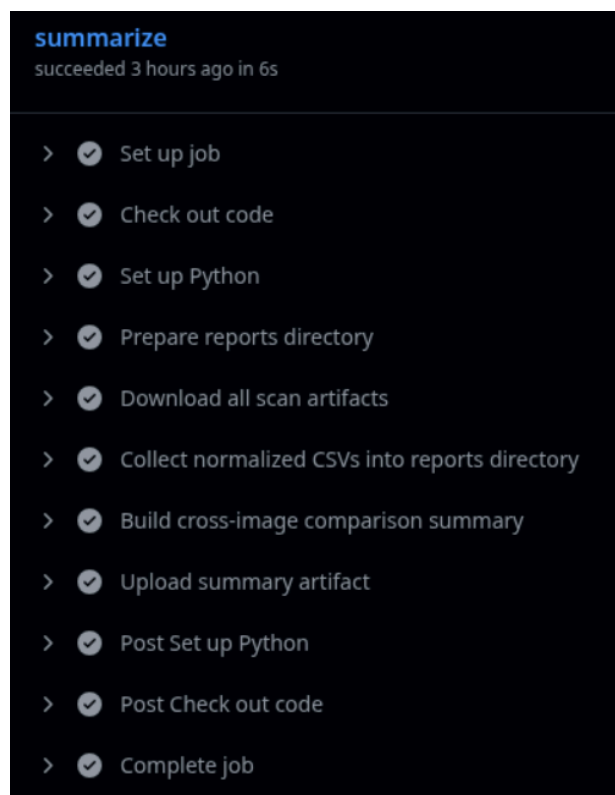


Figure 4: Screenshot showing the workflow's sequence of steps to summarize results

The data/reporting model focuses on how scan outputs are transformed for analysis. Each CI run produces two primary JSON artifacts per image: one from Trivy and one from Gype. A set of Python scripts in the repository parse these JSON files, extract comparable fields (such as

vulnerability identifier, package name, version, severity), and output normalised CSV files for each tool. A comparison script then joins these normalised datasets to identify overlaps and differences between the scanners' findings on the same image, and computes simple metrics such as counts per severity, overlap ratio, and examples of disagreements. These CSV files form the basis for the quantitative tables and charts used later in the Findings chapter, and can be regenerated from stored artifacts to reproduce the analysis.

Planning

Technology selection

The technologies used in this project were chosen as they meet the practical requirements for the scale of this project: they are open source, widely adopted in industry, well documented, and suitable for demonstrating a realistic container security and GitOps workflow. Furthermore, the stack needed to be lightweight enough to run locally and in CI, while still reflecting the kinds of tools commonly used in modern DevOps and DevSecOps environments.

Github Actions

GitHub Actions was selected for CI because it is tightly integrated with the source repository, supports matrix-based workflows, and is widely used for automated build and test pipelines. It is also well documented and easy to extend, which makes it a strong fit for a project that needs to build container images, run security scans, and archive reports as part of the same workflow. In practical terms, it reduces setup overhead and keeps the automation close to the codebase.

GitHub Container Registry

GitHub Container Registry was chosen as the image registry because it integrates naturally with GitHub repositories and workflows. It supports storing versioned container images alongside the project, which simplifies the handoff between build and deployment stages. Using GHCR also avoids introducing an additional external registry service, keeping the project architecture simpler and easier to reproduce.

kind

kind was used to provide a local Kubernetes cluster for development and experimentation. It is lightweight, easy to create and destroy, and ideal for testing Helm charts and GitOps workflows without requiring a full cloud Kubernetes environment. This made it especially suitable for repeatable local testing and for demonstrating deployment behaviour in a controlled setting.

Helm

Helm was selected because it provides a standard way to package Kubernetes resources into reusable charts. This is useful in the project because the application needs to be deployed consistently across environments, while still allowing configuration such as image tags, replica counts, and service settings to be changed through values files. Helm also fits naturally with ArgoCD, since ArgoCD can render and manage Helm charts directly.

ArgoCD

ArgoCD was chosen to implement the GitOps approach because it continuously reconciles the desired state in Git with the live state in the cluster. This is useful for demonstrating how deployment changes can be tracked, reviewed, and applied automatically from version control. It also helps make the deployment process more transparent, since the repository becomes the source of truth for what should be running in the cluster.

Trivy and Gype

Trivy and Gype were selected as the two vulnerability scanners because both are open source, widely used, and provide good documentation and machine-readable output. Using two scanners strengthens the analysis because it allows comparison of detection quality, overlap, and noise rather than relying on a single tool's results. They also fit the project's requirements well because they can be run non-interactively in CI and their JSON output can be consumed by later analysis stages.

Phased Plan

- P1: Build app, Dockerfile, basic tests.
- P2: Add Trivy and Gype locally, normalize outputs.
- P3: Container registry, Kubernetes, Helm.
- P4: ArgoCD, GitOps deployment.
- P5: CI workflows and experiments comparing scanners.

Implementation

Application Implementation

The application is a small FastAPI service that provides a realistic Python web workload for the containerisation pipeline. It is a web application with a Large Language Model (LLM) wrapper from Ollama used to summarize long-winded policy or compliance documents for people to see what is being required of them without having to read all the small print.

It is named 'Terms Long; Didn't Read' or 'TL;DR' and to keep it lightweight, the llama3 model was pulled to facilitate it. FastAPI was chosen because it is also lightweight, modern, and well suited to building simple API endpoints such as `/health` and `/test`, which makes it useful for demonstrating build, scan, and deployment behaviour without introducing unnecessary application complexity. The project uses the application primarily as a repeatable container image target rather than as a feature-complete service, though AI was used to produce a simple UI to complement it.

The screenshot shows a web application titled "Terms Long; Didn't Read". Below the title is a subtitle: "Paste policy, compliance, contract, or regulatory text and get structured outputs: obligations, required data, deadlines, who it applies to, and unclear points." The main input area is labeled "Text to analyse" and contains a sample text block: "This website uses cookies to monitor browsing preferences. If you do allow cookies to be used, the following personal information may be stored by us for use by third parties". Below the input area is a green button labeled "Analyse text" with a subtitle: "Best for requirement-heavy text, policies, and compliance materials." The output is displayed in four panels: "Obligations" (allow cookies to be used), "Required Data" (insert list of information), "Deadlines" (Nothing clearly identified.), and "Applies To" (website users).

Obligations	Required Data
allow cookies to be used	insert list of information

Deadlines	Applies To
Nothing clearly identified.	website users

Figure 5: UI of 'Terms Long; Didn't Read' - a FastAPI/Llama3 web application for policy summaries.

Dockerfile

The Dockerfile defines how the FastAPI application is packaged into a container image. In line with common container security practice, the Dockerfile should favour a minimal base image, install only the dependencies required to run the service, and avoid running the application as root where possible. These choices reduce attack surface and make the resulting image more suitable for vulnerability scanning and deployment in an automated pipeline.

Container Registry and Image Management

GitHub Container Registry (GHCR) is used to store built images under the project's namespace, allowing the container image lifecycle to remain closely tied to the GitHub repository and its workflows. In this project, image tags should identify specific builds rather than relying only on *latest*, as commit-based tags make deployments easier to trace and reproduce. This is particularly useful when the deployment pipeline later consumes the image tag from the manifests repository.

Kubernetes and Helm

The application is deployed to a local *kind* cluster, which is designed for running Kubernetes clusters inside Docker containers and is well suited to local development and CI experimentation. Helm is used to package the Kubernetes resources required by the application into a reusable chart, typically including metadata in *Chart.yaml* and templates for the Deployment and Service resources. This structure allows the project to manage application configuration declaratively while keeping the deployment artefacts aligned with version control.

ArgoCD and GitOps Setup

ArgoCD treats Git as the source of truth for what should be running in the cluster. When I commit changes to the *tldr-project* repository (for example to the *tldr-app-chart* Helm chart or its *values.yaml*) and push to the tracked branch (*main*), ArgoCD automatically notices that Git's desired state has changed. It then renders the Helm chart into Kubernetes manifests and compares those manifests with the live state of the *tldr* namespace. If there are differences (for example, a new image tag, a different replica count, or a modified Service), ArgoCD marks the application as *OutOfSync* and performs a sync to apply the changes to the cluster.

Enabling *selfHeal: true* means that if someone manually changes or deletes objects in the cluster with *kubectl* and those changes don't match what is defined in Git, ArgoCD will detect

the 'drift' during its reconciliation loop and automatically re-apply the Git version, restoring the application to the desired state without manual intervention.

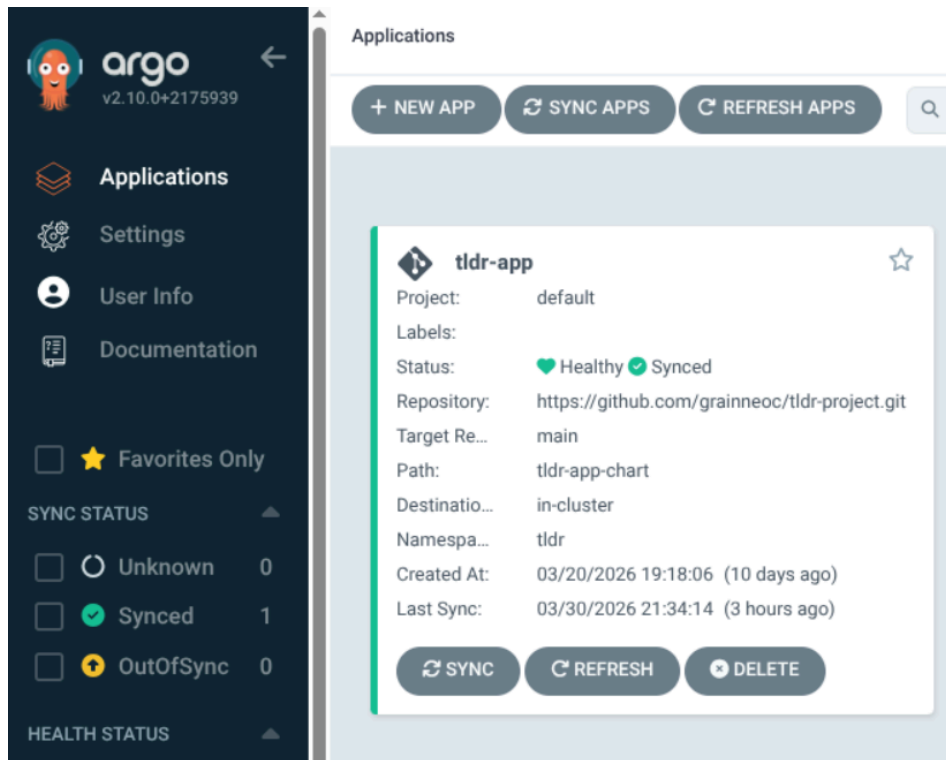


Figure 6: Screenshot of ArgoCD dashboard showing tldr-app in healthy and synced state.

CI Workflows

The GitHub Actions workflow incorporates a matrix strategy over the three Docker Hub test images (*alpine:3.18*, *debian:12* and *python:3.14-slim*), with an additional entry later for the hardened Python base. This 'matrix' strategy is documented in a technical blog (Oneuptime, 2025) which laid out the template and guide to implementation.

For each matrix entry, the job pulls it from Docker Hub (public registry), then runs Trivy and Gype in sequence against the resulting image tag. Each scanner produces a JSON report, and both reports are uploaded as artifacts so they can be normalised into CSV, compared, and reused in the Findings chapter without rerunning the scans.

This was evolved from the first instance of the CI workflow which was tailored to the tldr-app image but did not facilitate the reusability required for testing pulled public images from Docker Hub.

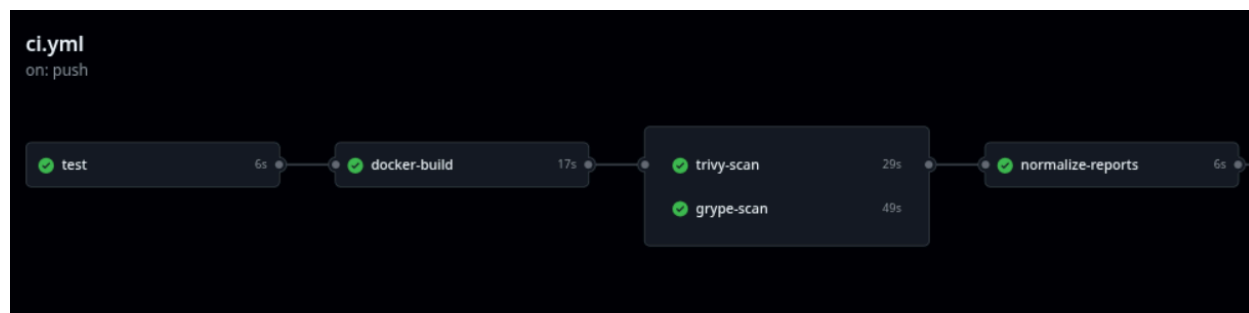


Figure 7: Screenshot of the sequence of jobs in the tldr-app workflow.

Security Scanning

Trivy is used to scan each image for vulnerabilities and can emit JSON reports with severity data and package metadata suitable for downstream analysis. Grype is used in the same way, also providing JSON output that exposes match-level vulnerability information for comparison with Trivy's results. Using both tools in the same workflow allows the project to compare not only the number of findings, but also overlap, severity distribution, and differences in how each scanner reports the same image.

```
{
  "SchemaVersion": 2,
  "CreatedAt": "2026-03-28T17:21:31.568399086Z",
  "ArtifactName": "tldr-app:ci-build",
  "ArtifactType": "container_image",
  "Metadata": {
    "Size": 171791872,
    "OS": {
      "Family": "debian",
      "Name": "13.4"
    }
  },
  "ImageID": "sha256:9bda805ddb0df6ec256b543f76101f3efd1b3966a8148f911f5d033e07a310",
  "DiffIDs": [
    "sha256:188c9b34dfbe022075d01fc4f5a305412909ef97de440783c15043e68e1b1913",
    "sha256:7c3d855adb27d35152846843945ba4762b4f29c81803982b791f04a4ccbdba0d",
    "sha256:00111f9d6e574635f3d8936186a797c237a5e117c6a70e7968bee027239da25f",
    "sha256:21f42d92f0f2265871e6f3f359987b097000d3a812d93ad56fc6218864c570b8",
    "sha256:0a19acf947bbdb6e932fab0525589d7a1ade129d83739923e14dc085f8c79bcb",
    "sha256:f52c186ba3feae9522522f58ba08c9b8e580f4e942340bff0bb579fe2bbeef4e",
    "sha256:40753fa6c8f9ec8bc3040ea325459029fe03afb7725236bef62a6a0701303b7b",
    "sha256:99be1bc4d2aef92332cb33ad80e93679b891a0398345fe22859a59f565269927"
  ],
  "RepoTags": [
    "tldr-app:ci-build",
    "ghcr.io/grainneoc/tldr-app:latest"
  ]
}
```

Figure 8: Snippet of one of the Trivy JSON reports (not normalized)

Findings

This study evaluates Trivy and Gype on four container images:

- *tldr-app:ci-build* (project application image).
- *alpine:3.18* (minimal base image).
- *debian:12* (general-purpose base image).
- *python:3.14-slim* (Python runtime image on a Debian base).

Initially the intention was to scan the tldr-app image in the same workflow matrix job as the Docker Hub images. However, this created issues in the CI run and became difficult to do in one workflow. As the CI/CD pipeline is configured to run the tldr application, it was simpler and neater to let the CI workflow for that remain unchanged as it builds the image and follows the full pipeline with each run.

The sample scans from Docker Hub required a 'docker pull' rather than 'docker build' so it was a straight forward process to place them together in a separate matrix job, while still running the same scan on them - just not pushing to GitHub Container Registry when it passes the checks, as happens with the tldr application. Each workflow builds or pulls the image, runs Trivy and Gype to produce JSON reports, and uploads these reports as artifacts for later analysis.

Detection quality tests

For each image and tool, the analysis collects:

- Total number of reported vulnerabilities.
- Counts per severity level (Critical, High, Medium, Low).
- The overlap between tools: vulnerabilities reported by both scanners versus those reported by only one.

To approximate ground truth, a small sample of findings per image is manually inspected and categorised as:

- Confirmed true positives (real, relevant vulnerabilities).
- Clear false positives (reported but not actually exploitable in this context).
- Likely false negatives (real vulnerabilities seen by one tool but missing in the other's output).

Actionability tests

For a subset of confirmed real findings from each tool, the study rates with a score from 1-5:

- Clarity of description (is the issue understandable without extra research?).
- Presence and usefulness of remediation advice.
- Helpfulness of severity and metadata (such as links or identifiers) for prioritisation.

These factors are combined into a simple "actionability" score for each finding and recorded in the *Actionability Scores* table under *Qualitative Findings*.

4 - 5 = High actionability (HA)

3 = Moderate actionability (MA)

1 - 2 = Low actionability / noisy (LA)

Developer experience tests

Using data from repeated CI runs, the study records:

- Scan time per image and per tool, and the overall impact on pipeline duration.
- Any configuration issues or integration effort required to run each scanner in GitHub Actions.

In this project the scanners always exit successfully on completed scans and do not directly enforce policy. A separate Policy Gate stage is designed to consume the normalised outputs and apply thresholds (for example, failing on High or Critical issues), but this gate is kept in observation mode during the experiments, so failures represent technical errors rather than enforced security policy.

Quantitative Findings

		Severity Rating							
Image	Tool	Critical	High	Medium	Low	Neg.	Unknown	Total	Overlap
tldr-app:ci-build	Trivy	0	6	9	71	n/a	n/a	86	
tldr-app:ci-build	Grype	0	10	19	5	47	9	93	
alpine:3.18	Trivy	0	0	0	0	n/a	n/a	0	0
alpine:3.18	Grype	0	0	4	6	0	0	10	
debian:12	Trivy	1	7	21	71	n/a	n/a	100	40
debian:12	Grype	1	8	13	3	54	9	89	
python:3.14-slim	Trivy	0	6	7	71	n/a	n/a	84	24
python:3.14-slim	Grype	0	6	10	3	49	10	78	

Table 4: Total number of vulnerabilities, severities and overlap of Trivy and Grype taken from scan results.

The above table is a summary of the quantitative results of Trivy and Gype's performance on the sample set of container images. It shows the number of vulnerabilities detected under each severity. Results from the Python script to capture the vulnerabilities detected by both Trivy and Gype are shown in the Overlap column. Trivy appears to have a lot of duplication of vulnerabilities which may account in some part for the disparity in numbers of LOW severity findings in particular.

```
▼ ✓ Compare current image Trivy vs Gype

1  ▶ Run python scripts/compare.py
11 Total Trivy CVEs: 47
12 Total Gype CVEs: 41
13 Overlap (in both): 40
14 Only in Trivy: 7
15 Only in Gype: 1
16
17 Examples only in Trivy:
18   CVE-2026-4105
19   CVE-2026-29111
20   CVE-2025-69720
21   TEMP-0841856-B18BAF
22   TEMP-0517018-A83CE6
23   CVE-2026-4438
24   CVE-2026-4437
25
26 Examples only in Gype:
27   CVE-2026-2219
```

Figure 8: Comparison results as visible from the GitHub Action Workflow view

tool	image	target	vuln_id	severity
gype	debian:12	debian:12	CVE-2011-3374	NEGLIGIBLE
gype	debian:12	debian:12	CVE-2025-14104	NEGLIGIBLE
gype	debian:12	debian:12	CVE-2022-0563	NEGLIGIBLE
gype	debian:12	debian:12	CVE-2026-3184	UNKNOWN
gype	debian:12	debian:12	CVE-2016-2781	LOW
gype	debian:12	debian:12	CVE-2025-5278	NEGLIGIBLE
gype	debian:12	debian:12	CVE-2017-18018	NEGLIGIBLE
gype	debian:12	debian:12	CVE-2025-6297	HIGH
gype	debian:12	debian:12	CVE-2026-2219	UNKNOWN
gype	debian:12	debian:12	CVE-2022-27943	NEGLIGIBLE
gype	debian:12	debian:12	CVE-2025-68972	MEDIUM
gype	debian:12	debian:12	CVE-2025-30258	MEDIUM
gype	debian:12	debian:12	CVE-2022-3219	NEGLIGIBLE
gype	debian:12	debian:12	CVE-2011-3374	NEGLIGIBLE
gype	debian:12	debian:12	CVE-2025-14104	NEGLIGIBLE
gype	debian:12	debian:12	CVE-2022-0563	NEGLIGIBLE
gype	debian:12	debian:12	CVE-2026-3184	UNKNOWN
gype	debian:12	debian:12	CVE-2026-0915	HIGH
gype	debian:12	debian:12	CVE-2026-0861	HIGH
gype	debian:12	debian:12	CVE-2025-15281	HIGH
gype	debian:12	debian:12	CVE-2010-0102	NEGLIGIBLE

Figure 9: Normalized Gype scan results of debian:12 image



trivy	debian:12	debian:12 (debian 12.13)	CVE-2024-56433	LOW
trivy	debian:12	debian:12 (debian 12.13)	CVE-2022-0563	LOW
trivy	debian:12	debian:12 (debian 12.13)	CVE-2025-14104	LOW
trivy	debian:12	debian:12 (debian 12.13)	CVE-2026-3184	LOW
trivy	debian:12	debian:12 (debian 12.13)	CVE-2025-69720	HIGH
trivy	debian:12	debian:12 (debian 12.13)	CVE-2023-50495	MEDIUM
trivy	debian:12	debian:12 (debian 12.13)	CVE-2025-6141	LOW
trivy	debian:12	debian:12 (debian 12.13)	CVE-2025-69720	HIGH
trivy	debian:12	debian:12 (debian 12.13)	CVE-2023-50495	MEDIUM
trivy	debian:12	debian:12 (debian 12.13)	CVE-2025-6141	LOW
trivy	debian:12	debian:12 (debian 12.13)	CVE-2007-5686	LOW

Figure 10: Normalized Trivy scan results of debian:12 image

Qualitative Findings

Configuration and integration complexity

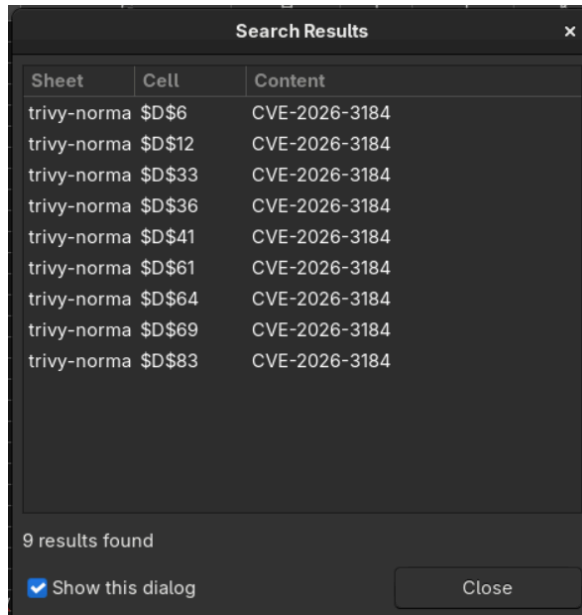
Both Gype and Trivy were very straightforward to integrate into the CI workflow. They both come with many options to satisfy policy implementation which was used both locally and in the pipeline throughout development before a specific policy script was created. They both can be configured to show fixed versions of vulnerabilities if available, along with a host of other flags for bespoke setups. In the context of GitHub Actions, there was ample documentation and availability of templates of a wide array of scenarios making the configuration broadly seamless and fast.

In terms of integration and configuration, for the purposes of this small-scale project, both were relatively simple to implement.

Noise and predictability

Trivy at first glance appeared to be the more stable of the two tools, and easier to predict as the severity labels were fewer. Having a label which states 'UNKNOWN', which Gype uses, is not the most helpful, nor is having to decipher between 'LOW' and 'NEGLIGIBLE' severities. While Gype gives more detail by way of more severity types, perhaps, it is not necessarily useful to the end user.

However, CVE-2026-29111 was detected by Trivy during the scan of *python:3.14-slim* and rated severity 'HIGH' yet, references a version of the *systemd* package far older than the one used by this image. This is a false positive result with severity 'HIGH' which is very misleading for users. Furthermore, Trivy is very noisy with many duplicates of results as shown in the screenshot below. This skews the true number of vulnerabilities and requires further configuration to omit duplicate results.



The screenshot shows a 'Search Results' dialog box with a table of 9 results. All results are duplicates of CVE-2026-3184. The dialog includes a 'Close' button and a checkbox for 'Show this dialog' which is checked.

Sheet	Cell	Content
trivy-norma	\$D\$6	CVE-2026-3184
trivy-norma	\$D\$12	CVE-2026-3184
trivy-norma	\$D\$33	CVE-2026-3184
trivy-norma	\$D\$36	CVE-2026-3184
trivy-norma	\$D\$41	CVE-2026-3184
trivy-norma	\$D\$61	CVE-2026-3184
trivy-norma	\$D\$64	CVE-2026-3184
trivy-norma	\$D\$69	CVE-2026-3184
trivy-norma	\$D\$83	CVE-2026-3184

9 results found

☒ Show this dialog Close

Figure 11: Duplication of Trivy results for CVE-2026-3184

Clarity and remediation guidance

Vulnerability ID	Severity	Result	Score (1-5)
<i>python:3.14-slim</i>			
CVE-2025-6141	Trivy: LOW Grype: MEDIUM	False Positive	Grype: 1 Trivy: 2
CVE-2026-27171	MEDIUM	True Positive	Grype: 4 Trivy: 4
CVE-2024-56433	LOW	False Positive (context)	Grype: 2 Trivy: 2
CVE-2025-14104	Trivy: LOW Grype: NEGLIGIBLE	False Positive (context)	Grype: 3 Trivy: 2
CVE-2026-3184	Trivy: LOW Grype: UNKNOWN	False Positive (context)	Grype: 1 Trivy: 2
<i>debian:12</i>			
CVE-2023-45853	CRITICAL	False Positive (context)	Grype: 1 Trivy: 1
CVE-2026-0861	HIGH	False Positive (Functional)	Grype: 3 Trivy: 3
CVE-2025-15281	MEDIUM	True Positive/ Functional False	Grype: 4 Trivy: 4
CVE-2011-3389	LOW	True Positive/ Functional False	Grype: 2 Trivy: 2
<i>alpine:3.18</i>			
No overlap of vulnerabilities in results			
<i>tldr-app:ci-build</i>			
CVE-2025-69720	HIGH	False Positive (context)	Grype: 1 Trivy: 1
CVE-2026-4105	MEDIUM	True Positive/ Functional False	Grype: 4 Trivy: 4
CVE-2026-2673	LOW	True Positive	Grype: 5 Trivy: 5

Table 5: sample set of overlap vulnerabilities

The table showing the overlap of vulnerabilities across a small sample set is discussed below, with some examples setting out where the information in each column comes from.

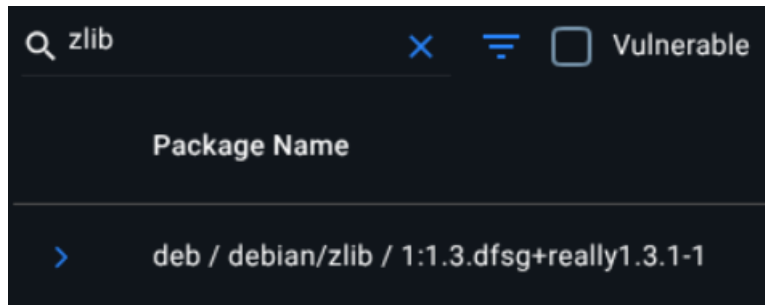


Figure 12: Debian package 'zlib' subject of CVE-2023-45853



Figure 13: Screenshot from National Vulnerability Database showing affected software for CVE-2023-45853

As shown in the screenshots above for CVE-2023-45853 - which was flagged as CRITICAL for both Trivy and Gripe, the affected package 'zlib', is currently on version 1.3.1 which is shown by National Vulnerability Database to be excluded from the impact of this package. This shows that both Trivy and Gripe incorrectly gave this a severity of CRITICAL, even though the package is already updated to the patched version. This depicts what is referred to in the Overlap of Vulnerabilities table as a 'false positive', highlighted here especially as it had the highest severity yet was still inaccurate.

The use of (*context*) is to demonstrate that this is a potential threat, but either not in the environment of this project or highly unlikely to be triggered by the context of the project. For example, some of the results here, on inspection, are local environment threats at a system level, as opposed to the remote container environment of these scans.

Further to this however, this vulnerability CVE-2025-15281 which is described in the table above as '*True Positive/Functional False*', means that while the bug does exist in the library in question, it is very only under very specific circumstances that this could be exploited so is not a priority. For MEDIUM severity, both Trivy and Gripe score a 4 here as this is actionable by updating to the next version of the image when it is released, is unlikely to affect the environment, but has a known issue.

From analysing these results, while scans can be configured to produce very useful information such as whether fixes exist for certain vulnerabilities, Trivy and Gype both require extended configuration to form usable results that developers can act quickly on.

Actionability Scores	
Trivy	32
Gype	31

Table 6: Actionability Scores of Trivy and Gype on a sample-set of vulnerabilities.

Across the manually reviewed sample, Trivy and Gype achieved very similar actionability scores (32 vs 31 out of a possible total based on the number of rated findings), indicating that, on average, their real findings were equally usable for developers in terms of clarity, remediation hints, and prioritisation metadata.

Given the minimal difference between totals, this study does not claim one scanner is meaningfully more actionable than the other; instead, both cluster around the same 'moderate-high' actionability band on the 1–5 scale.

Scan Times

In previous working versions of the project, Python was used to capture output of scan times in seconds. When building locally this made sense. However, as the project evolved, it was simpler to look at the time as it is recorded in the GitHub Actions Workflow. As shown in the screenshot below, the time for each image to be pulled from the registry, scanned by both Trivy and Gype, vulnerabilities compiled into JSON and converted to CSV is all a matter of seconds. For this reason, this project finds that scan times are a positive feature of both tools which would benefit all end users.

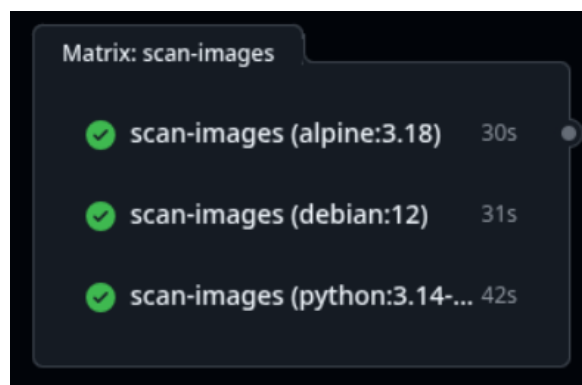


Figure 13: Screenshot of image scan times

Reflection

What Worked Well

GitOps with ArgoCD and Helm

A key strength of the project was the use of GitOps with ArgoCD and Helm. This combination created a clear and repeatable deployment model in which the desired state of the application was defined in Git and then reconciled automatically in the cluster. Helm made the Kubernetes manifests easier to parameterise and reuse, while ArgoCD provided a straightforward mechanism for observing drift and keeping the live environment aligned with the repository. Together, they reduced manual deployment effort and made changes easier to trace, review, and reproduce.

Using CI for scanning and comparison

Another aspect that worked particularly well was integrating vulnerability scanning into the continuous integration pipeline. Running the scanners automatically on a consistent set of images made the comparison more reliable and reduced the risk of ad hoc or inconsistent testing. It also allowed the project to collect structured JSON outputs that could be analysed later for severity, overlap, and quality differences. This approach made the scanning process repeatable and supported a more evidence-based comparison between the tools.

Persona-driven requirements

The use of persona-driven requirements also proved valuable because it kept the project grounded in real user needs rather than purely technical goals. By considering the perspectives of roles such as developer, DevOps engineer, DevSecOps practitioner, and product owner, it became easier to justify design choices and evaluate whether the system addressed practical concerns such as usability, automation, security visibility, and deployment reliability. This helped ensure that the project was not only technically functional but also relevant to different stakeholders.

Challenges and Limitations

GHCR authentication

One of the practical challenges was configuring authentication between the CI pipeline and GitHub Container Registry. Pulling and pushing images required the correct permissions, token handling, and registry setup, which introduced an additional layer of configuration complexity. While this is a common issue in container-based workflows, it can still be a source of failure if credentials, scopes, or repository settings are misconfigured.

Automated ArgoCD sync

I had difficulty getting the CD to happen automatically in Argo CD instead of only after a manual sync. At first I was only pushing a new image to GHCR, but Argo CD was watching the Git-tracked Helm values, and because the image tag in Git stayed the same, the cluster never changed on its own. To fix it, I changed the workflow so it tags each image with *github.sha*, updates *tldr-app-chart/values.yaml* with that same SHA, commits the change back to the repository, and then lets Argo CD auto-sync from the new Git commit.

Limited ground truth for vulnerability findings

A major limitation was the absence of a complete ground truth dataset for vulnerabilities in the scanned images. Because only a small sample of findings was manually inspected, the classification of true positives, false positives, and likely false negatives remains approximate rather than definitive. This means the reported quality comparisons should be interpreted as indicative evidence rather than a full audit of every vulnerability detected by either scanner.

Time constraints and scope

Time constraints also limited the number of images, scenarios, and repeated experiments that could be included. The study therefore focused on a small set of representative container images rather than a broader range of application types or base images. As a result, the findings are useful for illustrating relative tool behaviour in this project context, but they cannot be generalised confidently to all containerised workloads.

Due to time constraints, the deployed Kubernetes service was not externally exposed. The Helm chart retained an internal *ClusterIP* service configuration, while both Ingress and HTTPRoute remained disabled, so public access testing of the live deployment was outside the final project scope.

Future Work

DAST or runtime scanning

A natural extension of this project would be to add dynamic application security testing or runtime scanning alongside the existing container image analysis. DAST would allow the application to be exercised in a running state and tested for issues that static image scanning cannot detect, such as insecure endpoints, misconfigurations, or unexpected response behaviour. Runtime scanning would also help capture vulnerabilities or policy violations that only appear after deployment, making the evaluation more representative of a real production environment.

Add more scanners

Another useful direction would be to expand the comparison to include additional vulnerability scanners. This would make it possible to assess whether the differences observed between the current tools are tool-specific or part of a wider pattern across the scanning ecosystem. Adding more scanners could also improve the robustness of the findings by increasing coverage across different vulnerability databases, detection methods, and reporting styles.

Metrics and dashboards

The project could also be extended with richer metrics and a dashboard layer for visual analysis. At the moment, the results are primarily interpreted through tables and manual comparison, but a dashboard would make it easier to track trends such as scan time, severity distribution, overlap, and false positive rate across images and over time. This would improve the usability of the results for both technical and non-technical stakeholders, and it would make the pipeline more useful as a continuous monitoring tool rather than just an experimental benchmark.

SBOM generation

In this project, Trivy and Gype scanned container images directly, which is sufficient for comparing their detection quality and CI behaviour. In future work, the pipeline could be extended to generate and store Software Bills of Materials (SBOMs) for each image (for example using Syft or Trivy's SBOM features) and then run vulnerability scans against those SBOMs rather than the images themselves. This would enable reuse of a single SBOM across multiple tools, support rescanning as vulnerability databases evolve, and provide a more robust basis for supply-chain visibility and compliance reporting.

References

NOTE: References were reused for this report from a short-form report completed for a System Security assignment last semester.

- National Institute of Standards and Technology (NIST), Application Container Security Guide (NIST Special Publication 800-190), available at <https://nvlpubs.nist.gov/nistpubs/specialpublications/nist.sp.800-190.pdf> Accessed on 4th December, 2025.
- Cloud Native Now, The Evolution of Container Security, available at <https://cloudnativenow.com/topics/cloudnativesecurity/the-evolution-of-container-security> Accessed on 4th December, 2025.
- AWS (Amazon Web Services), What is Containerization?, available at <https://aws.amazon.com/what-is/containerization/> Accessed on 4th December, 2025.
- OWASP Foundation, Container Vulnerability Scanning, available at <https://owasp.org/www-project-devsecops-guideline/latest/02f-Container-Vulnerability-Scanning> Accessed on 4th December, 2025.
- Perplexity AI, Homepage, available at <https://www.perplexity.ai/> Accessed on 4th-8th December, 2025.
- ARMO, What is Container Image Scanning - Explanation & Best Practices for Images Security (2025), available at <https://www.armosec.io/glossary/container-image-scanning/> Accessed on 5th-8th December, 2025.
- Docker Docs, Docker Engine, available at <https://docs.docker.com/engine/> Accessed on 5th December, 2025.
- Clair Documentation (Quay), What is ClairV4, available at <https://quay.github.io/clair/whatis.html> Accessed on 6th December, 2025.
- Docker Blog, 2025 Docker State of App Dev: Key Insights Revealed, available at <https://www.docker.com/blog/2025-docker-state-of-app-dev/> Accessed on 6th December, 2025.
- Alpine Linux, about, available at <https://alpinelinux.org/about/> Accessed on 7th December, 2025.
- Docker Hub, vulnerables/web-dvwa (Image Repository), available at <https://hub.docker.com/r/vulnerables/web-dvwa> Accessed on 7th December, 2025.

- Snyc (no date) *Shift-left security: what it is and how to do it right*. Available at: <https://snyk.io/articles/shift-left-security/> (Accessed: 31 March 2026).
- GoCodeo (no date) *Grype: A vulnerability scanner for container images and filesystems*. Available at: <https://www.gocodeo.com/post/grype-a-vulnerability-scanner-for-container-images-and-filesystems> (Accessed: 31 March 2026).
- Jit (no date) *A guide to generating SBOM with Syft and Grype*. Available at: <https://www.jit.io/resources/appsec-tools/a-guide-to-generating-sbom-with-syft-and-grype> (Accessed: 31 March 2026).
- Docker (no date) *Docker Hardened Images documentation*. Available at: <https://docs.docker.com/dhi/> (Accessed: 31 March 2026).
- OneUptime (2025) *Container scanning in GitHub Actions* [Blog]. Available at: <https://oneuptime.com/blog/post/2025-12-20-container-scanning-github-actions/view> (Accessed: 31 March 2026).
- Python Software Foundation (no date) *json — JSON encoder and decoder. Python 3 documentation*. Available at: <https://docs.python.org/3/library/json.html> (Accessed: 31 March 2026).
- National Institute of Standards and Technology (no date) *National Vulnerability Database (NVD)*. Available at: <https://nvd.nist.gov/> (Accessed: 31 March 2026).

Appendices

AI use

- Troubleshooting FastAPI/Ollama application
- Full implementation of the front-end of the application
- Python scripts for comparing vulnerability data
- Checking over CI templates
- Guidance on structure of report and sequence of implementation
- Implementation guidance for Kubernetes & ArgoCD
- Explanation of concepts such as the Vulnerability descriptions on National Vulnerability Database